Experiment 'p_14_2_selection_sort_interval_array' Results

June 6, 2026

Experiment outcome: FAILURE_VERIFY
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_14_2_selection_sort_interval_array
Natural language statement: Write a method that implements the selection sort algorithm to sort an array of values from a finite interval.
Method signature: p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted: array<int>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

Requires

- `forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that implements the selection sort algorithm to sort an array of values
    ↪ from a finite interval.

The signature should be:

p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted:
    ↪ array<int>)

The method should respect the following contract:

requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound, ensures forall i
    ↪ , j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures multiset(
    ↪ inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnkFGE6RArQtKcQA4kz0wz7gbR2hm
Model: gpt-5.5-2026-04-23
Created at: 1780748210
Finish reason: stop
Usage: 380 tokens in, and 11348 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var c := 0;
  while c < n
    invariant 0 <= c <= n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
    decreases n - c
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }

  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
```

```
assert forall k | 0 <= k < |sorted[..]|
  ensures sorted[..][k] == inputs[..][k]
{
  assert |sorted[..]| == n;
  assert |inputs[..]| == n;
  assert sorted[..][k] == sorted[k];
  assert inputs[..][k] == inputs[k];
}
assert sorted[..] == inputs[..];

var i := 0;
while i < n
  invariant sorted.Length == n
  invariant inputs.Length == n
  invariant 0 <= i <= n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
  decreases n - i
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i < n
    invariant i <= min < n
    invariant i + 1 <= j <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases n - j
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert forall k :: i <= k < j ==> sorted[oldMin] <= sorted[k];
      min := j;
      assert forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k == j {
        } else {
          assert i <= k < j;
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[k];
        }
      }
    } else {
      assert sorted[min] <= sorted[j];
      assert forall k | i <= k < j + 1
```

```
      ensures sorted[min] <= sorted[k]
    {
      if k == j {
      } else {
        assert i <= k < j;
      }
    }
  }
  j := j + 1;
}

ghost var pre := sorted[..];
assert |pre| == n;
assert multiset(pre) == multiset(inputs[..]);

assert forall k | i <= k < n
  ensures pre[min] <= pre[k]
{
  assert pre[min] == sorted[min];
  assert pre[k] == sorted[k];
}
assert forall p, q | 0 <= p < q < i
  ensures pre[p] <= pre[q]
{
  assert pre[p] == sorted[p];
  assert pre[q] == sorted[q];
}
assert forall p, q | 0 <= p < i <= q < n
  ensures pre[p] <= pre[q]
{
  assert pre[p] == sorted[p];
  assert pre[q] == sorted[q];
}

if min != i {
  assert i < min;
  sorted[i], sorted[min] := sorted[min], sorted[i];

  ghost var swapped := pre[i := pre[min]][min := pre[i]];
  assert forall k | 0 <= k < |sorted[..]|
    ensures sorted[..][k] == swapped[k]
  {
    assert |sorted[..]| == n;
    assert sorted[..][k] == sorted[k];
    if k == i {
      assert sorted[k] == pre[min];
      assert swapped[k] == pre[min];
    } else if k == min {
      assert sorted[k] == pre[i];
      assert swapped[k] == pre[i];
    } else {
```

```
      assert sorted[k] == pre[k];
      assert swapped[k] == pre[k];
    }
  }
  assert sorted[..] == swapped;

  ghost var A := pre[..i];
  ghost var B := pre[i + 1..min];
  ghost var C := pre[min + 1..];
  assert pre == A + [pre[i]] + B + [pre[min]] + C;
  assert swapped == A + [pre[min]] + B + [pre[i]] + C;
  calc {
    multiset(swapped);
  == {
    assert swapped == A + [pre[min]] + B + [pre[i]] + C;
  }
    multiset(A + [pre[min]] + B + [pre[i]] + C);
  == {
  }
    multiset(A) + multiset([pre[min]]) + multiset(B) + multiset([pre[i]]) + multiset(C
      ↪ );
  == {
  }
    multiset(A) + multiset([pre[i]]) + multiset(B) + multiset([pre[min]]) + multiset(C
      ↪ );
  == {
  }
    multiset(A + [pre[i]] + B + [pre[min]] + C);
  == {
    assert pre == A + [pre[i]] + B + [pre[min]] + C;
  }
    multiset(pre);
  }
  assert multiset(sorted[..]) == multiset(pre);
} else {
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[..] == pre[i := pre[min]][min := pre[i]];
assert sorted[i] == pre[min];
assert sorted[min] == pre[i];
assert forall k | 0 <= k < n && k != i && k != min
  ensures sorted[k] == pre[k]
{
  assert sorted[..][k] == (pre[i := pre[min]][min := pre[i]])[k];
}
assert multiset(sorted[..]) == multiset(inputs[..]);

assert forall q | i + 1 <= q < n
  ensures sorted[i] <= sorted[q]
```

```
{
  assert sorted[i] == pre[min];
  if q == min {
    assert sorted[q] == pre[i];
    assert pre[min] <= pre[i];
  } else {
    assert sorted[q] == pre[q];
    assert pre[min] <= pre[q];
  }
}

assert forall p, q | 0 <= p < q < i + 1
  ensures sorted[p] <= sorted[q]
{
  if q < i {
    assert sorted[p] == pre[p];
    assert sorted[q] == pre[q];
    assert pre[p] <= pre[q];
  } else {
    assert q == i;
    assert p < i;
    assert sorted[p] == pre[p];
    assert sorted[i] == pre[min];
    assert pre[p] <= pre[min];
  }
}

assert forall p, q | 0 <= p < i + 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if p < i {
    assert sorted[p] == pre[p];
    if q == min {
      assert sorted[q] == pre[i];
      assert pre[p] <= pre[i];
    } else {
      assert sorted[q] == pre[q];
      assert pre[p] <= pre[q];
    }
  } else {
    assert p == i;
    assert sorted[i] == pre[min];
    if q == min {
      assert sorted[q] == pre[i];
      assert pre[min] <= pre[i];
    } else {
      assert sorted[q] == pre[q];
      assert pre[min] <= pre[q];
    }
  }
}
```

```
    i := i + 1;
  }

  assert i == n;
  assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_2_selection_sort_interval_array/program.dfy(33,2): Error: invalid
    ↪ UnaryExpression
   |
33 | while i < n
   | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(32,12): Error: expected
    ↪ either 'by' or a semicolon following the assert expression
   |
32 | var i := 0;
   | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(70,6): Error: invalid
    ↪ UnaryExpression
   |
70 | } else {
   | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(69,8): Error: expected
    ↪ either 'by' or a semicolon following the assert expression
   |
69 | }
   | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(80,6): Error: invalid
    ↪ UnaryExpression
   |
80 | }
   | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(79,8): Error: expected
    ↪ either 'by' or a semicolon following the assert expression
   |
79 | }
   | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(107,16): Error: then
    ↪ expected
```

```
    |
107 | if min != i {
    | ^

problem_data/p_14_2_selection_sort_interval_array/program.dfy(109,29): Error: rbrace
    ↪ expected
    |
109 | sorted[i], sorted[min] := sorted[min], sorted[i];
    | ^

8 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnkIjfjos9eBVg4t5ZWeSVhFheYoy
Model: gpt-5.5-2026-04-23
Created at: 1780748425
Finish reason: stop
Usage: 3111 tokens in, and 3080 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var c := 0;
  while c < n
    invariant 0 <= c <= n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
    decreases n - c
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }

  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  forall k | 0 <= k < |sorted[..]|
    ensures sorted[..][k] == inputs[..][k]
  {
    assert |sorted[..]| == n;
    assert |inputs[..]| == n;
    assert sorted[..][k] == sorted[k];
    assert inputs[..][k] == inputs[k];
  }
  assert sorted[..] == inputs[..];
```

```
var i := 0;
while i < n
  invariant sorted.Length == n
  invariant inputs.Length == n
  invariant 0 <= i <= n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
  decreases n - i
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i < n
    invariant i <= min < n
    invariant i + 1 <= j <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases n - j
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert forall k :: i <= k < j ==> sorted[oldMin] <= sorted[k];
      min := j;
      forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k == j {
        } else {
          assert i <= k < j;
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[k];
        }
      }
    } else {
      assert sorted[min] <= sorted[j];
      forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k == j {
        } else {
          assert i <= k < j;
        }
      }
    }
    j := j + 1;
```

```
  }

  ghost var pre := sorted[..];
  assert |pre| == n;
  assert multiset(pre) == multiset(inputs[..]);

  forall k | i <= k < n
    ensures pre[min] <= pre[k]
  {
    assert pre[min] == sorted[min];
    assert pre[k] == sorted[k];
  }
  forall p, q | 0 <= p < q < i
    ensures pre[p] <= pre[q]
  {
    assert pre[p] == sorted[p];
    assert pre[q] == sorted[q];
  }
  forall p, q | 0 <= p < i <= q < n
    ensures pre[p] <= pre[q]
  {
    assert pre[p] == sorted[p];
    assert pre[q] == sorted[q];
  }

  if min != i {
    assert i < min;
    var tmp := sorted[i];
    sorted[i] := sorted[min];
    sorted[min] := tmp;

    ghost var swapped := pre[i := pre[min]][min := pre[i]];
    forall k | 0 <= k < |sorted[..]|
      ensures sorted[..][k] == swapped[k]
    {
      assert |sorted[..]| == n;
      assert sorted[..][k] == sorted[k];
      if k == i {
        assert sorted[k] == pre[min];
        assert swapped[k] == pre[min];
      } else if k == min {
        assert sorted[k] == pre[i];
        assert swapped[k] == pre[i];
      } else {
        assert sorted[k] == pre[k];
        assert swapped[k] == pre[k];
      }
    }
    assert sorted[..] == swapped;

    ghost var A := pre[..i];
```

```
  ghost var B := pre[i + 1..min];
  ghost var C := pre[min + 1..];
  assert pre == A + [pre[i]] + B + [pre[min]] + C;
  assert swapped == A + [pre[min]] + B + [pre[i]] + C;
  calc {
    multiset(swapped);
  == {
    assert swapped == A + [pre[min]] + B + [pre[i]] + C;
  }
    multiset(A + [pre[min]] + B + [pre[i]] + C);
  == {
  }
    multiset(A) + multiset([pre[min]]) + multiset(B) + multiset([pre[i]]) + multiset(C
        ↪ );
  == {
  }
    multiset(A) + multiset([pre[i]]) + multiset(B) + multiset([pre[min]]) + multiset(C
        ↪ );
  == {
  }
    multiset(A + [pre[i]] + B + [pre[min]] + C);
  == {
    assert pre == A + [pre[i]] + B + [pre[min]] + C;
  }
    multiset(pre);
  }
  assert multiset(sorted[..]) == multiset(pre);
} else {
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[..] == pre[i := pre[min]][min := pre[i]];
assert sorted[i] == pre[min];
assert sorted[min] == pre[i];
forall k | 0 <= k < n && k != i && k != min
  ensures sorted[k] == pre[k]
{
  assert sorted[..][k] == (pre[i := pre[min]][min := pre[i]])[k];
}
assert multiset(sorted[..]) == multiset(inputs[..]);

forall q | i + 1 <= q < n
  ensures sorted[i] <= sorted[q]
{
  assert sorted[i] == pre[min];
  if q == min {
    assert sorted[q] == pre[i];
    assert pre[min] <= pre[i];
  } else {
    assert sorted[q] == pre[q];
```

```
      assert pre[min] <= pre[q];
    }
  }

  forall p, q | 0 <= p < q < i + 1
    ensures sorted[p] <= sorted[q]
  {
    if q < i {
      assert sorted[p] == pre[p];
      assert sorted[q] == pre[q];
      assert pre[p] <= pre[q];
    } else {
      assert q == i;
      assert p < i;
      assert sorted[p] == pre[p];
      assert sorted[i] == pre[min];
      assert pre[p] <= pre[min];
    }
  }

  forall p, q | 0 <= p < i + 1 <= q < n
    ensures sorted[p] <= sorted[q]
  {
    if p < i {
      assert sorted[p] == pre[p];
      if q == min {
        assert sorted[q] == pre[i];
        assert pre[p] <= pre[i];
      } else {
        assert sorted[q] == pre[q];
        assert pre[p] <= pre[q];
      }
    } else {
      assert p == i;
      assert sorted[i] == pre[min];
      if q == min {
        assert sorted[q] == pre[i];
        assert pre[min] <= pre[i];
      } else {
        assert sorted[q] == pre[q];
        assert pre[min] <= pre[q];
      }
    }
  }

  i := i + 1;
}

assert i == n;
assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
```

```
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_14_2_selection_sort_interval_array/program.dfy(1,7): Error: Verification
    ↪ of 'p_14_2_selection_sort_interval_array' timed out after 31 seconds. (the limit
    ↪ can be increased using --verification-time-limit)
  |
1 | method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
    ↪ sorted: array<int>)
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnkK8wjA9RAQY8qXvBEp7qFXhbRnn
Model: gpt-5.5-2026-04-23
Created at: 1780748512
Finish reason: stop
Usage: 5114 tokens in, and 7456 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n](k => inputs[k]);

  forall k | 0 <= k < n
    ensures sorted[..][k] == inputs[..][k]
  {
    assert sorted[..][k] == sorted[k];
    assert inputs[..][k] == inputs[k];
  }
  assert sorted[..] == inputs[..];
  assert {:split_here} true;

  var i := 0;
  while i < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
```

```
  decreases n - i
{
  var min := i;
  var j := i + 1;

  while j < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i < n
    invariant i <= min < n
    invariant i + 1 <= j <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases n - j
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert forall k :: i <= k < j ==> sorted[oldMin] <= sorted[k];
      min := j;
      forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k != j {
          assert i <= k < j;
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[k];
        }
      }
    } else {
      forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k == j {
          assert sorted[min] <= sorted[j];
        } else {
          assert i <= k < j;
        }
      }
    }
    j := j + 1;
  }

  assert {:split_here} true;
  ghost var pre := sorted[..];
  assert |pre| == n;
  assert forall k :: i <= k < n ==> pre[min] <= pre[k];
  assert forall p, q :: 0 <= p < q < i ==> pre[p] <= pre[q];
  assert forall p, q :: 0 <= p < i <= q < n ==> pre[p] <= pre[q];
```

```
if min != i {
  assert i < min;
  var tmp := sorted[i];
  sorted[i] := sorted[min];
  sorted[min] := tmp;

  ghost var swapped := pre[i := pre[min]][min := pre[i]];
  forall k | 0 <= k < n
    ensures sorted[..][k] == swapped[k]
  {
    assert sorted[..][k] == sorted[k];
    if k == i {
      assert sorted[k] == pre[min];
      assert swapped[k] == pre[min];
    } else if k == min {
      assert sorted[k] == pre[i];
      assert swapped[k] == pre[i];
    } else {
      assert sorted[k] == pre[k];
      assert swapped[k] == pre[k];
    }
  }
  assert sorted[..] == swapped;

  ghost var A := pre[..i];
  ghost var B := pre[i + 1..min];
  ghost var C := pre[min + 1..];
  assert pre == A + [pre[i]] + B + [pre[min]] + C;
  assert swapped == A + [pre[min]] + B + [pre[i]] + C;
  calc {
    multiset(swapped);
  ==
    multiset(A + [pre[min]] + B + [pre[i]] + C);
  ==
    multiset(A) + multiset([pre[min]]) + multiset(B) + multiset([pre[i]]) + multiset(C
        ↪ );
  ==
    multiset(A) + multiset([pre[i]]) + multiset(B) + multiset([pre[min]]) + multiset(C
        ↪ );
  ==
    multiset(A + [pre[i]] + B + [pre[min]] + C);
  ==
    multiset(pre);
  }
  assert multiset(sorted[..]) == multiset(pre);
} else {
  assert sorted[..] == pre;
  assert multiset(sorted[..]) == multiset(pre);
}

assert {:split_here} true;
```

```
  assert sorted[..] == pre[i := pre[min]][min := pre[i]];
  assert sorted[i] == pre[min];
  assert sorted[min] == pre[i];
  forall k | 0 <= k < n && k != i && k != min
    ensures sorted[k] == pre[k]
  {
    assert sorted[..][k] == (pre[i := pre[min]][min := pre[i]])[k];
  }
  assert multiset(sorted[..]) == multiset(inputs[..]);

  forall q | i + 1 <= q < n
    ensures sorted[i] <= sorted[q]
  {
    if q == min {
      assert sorted[q] == pre[i];
      assert pre[min] <= pre[i];
    } else {
      assert sorted[q] == pre[q];
      assert pre[min] <= pre[q];
    }
  }

  forall p, q | 0 <= p < q < i + 1
    ensures sorted[p] <= sorted[q]
  {
    if q < i {
      assert sorted[p] == pre[p];
      assert sorted[q] == pre[q];
      assert pre[p] <= pre[q];
    } else {
      assert q == i;
      assert p < i;
      assert sorted[p] == pre[p];
      assert sorted[i] == pre[min];
      assert pre[p] <= pre[min];
    }
  }

  forall p, q | 0 <= p < i + 1 <= q < n
    ensures sorted[p] <= sorted[q]
  {
    if p < i {
      assert sorted[p] == pre[p];
      if q == min {
        assert sorted[q] == pre[i];
        assert pre[p] <= pre[i];
      } else {
        assert sorted[q] == pre[q];
        assert pre[p] <= pre[q];
      }
    } else {
```

```
      assert p == i;
      assert sorted[i] == pre[min];
      if q == min {
        assert sorted[q] == pre[i];
        assert pre[min] <= pre[i];
      } else {
        assert sorted[q] == pre[q];
        assert pre[min] <= pre[q];
      }
    }
  }

  i := i + 1;
 }

 assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Final Program

```
method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
    ↪ sorted: array<int>)
      requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound
      ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
      ensures multiset(inputs[..]) == multiset(sorted[..])
{
 var n := inputs.Length;
 sorted := new int[n](k => inputs[k]);

 forall k | 0 <= k < n
   ensures sorted[..][k] == inputs[..][k]
 {
   assert sorted[..][k] == sorted[k];
   assert inputs[..][k] == inputs[k];
 }
 assert sorted[..] == inputs[..];
 assert {:split_here} true;

 var i := 0;
 while i < n
   invariant sorted.Length == n
   invariant inputs.Length == n
   invariant 0 <= i <= n
   invariant multiset(sorted[..]) == multiset(inputs[..])
   invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
   invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
   decreases n - i
 {
   var min := i;
```

```
  var j := i + 1;

  while j < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i < n
    invariant i <= min < n
    invariant i + 1 <= j <= n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    decreases n - j
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert forall k :: i <= k < j ==> sorted[oldMin] <= sorted[k];
      min := j;
      forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k != j {
          assert i <= k < j;
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[k];
        }
      }
    } else {
      forall k | i <= k < j + 1
        ensures sorted[min] <= sorted[k]
      {
        if k == j {
          assert sorted[min] <= sorted[j];
        } else {
          assert i <= k < j;
        }
      }
    }
    j := j + 1;
  }

  assert {:split_here} true;
  ghost var pre := sorted[..];
  assert |pre| == n;
  assert forall k :: i <= k < n ==> pre[min] <= pre[k];
  assert forall p, q :: 0 <= p < q < i ==> pre[p] <= pre[q];
  assert forall p, q :: 0 <= p < i <= q < n ==> pre[p] <= pre[q];

  if min != i {
    assert i < min;
    var tmp := sorted[i];
```

```
    sorted[i] := sorted[min];
    sorted[min] := tmp;

    ghost var swapped := pre[i := pre[min]][min := pre[i]];
    forall k | 0 <= k < n
      ensures sorted[..][k] == swapped[k]
    {
      assert sorted[..][k] == sorted[k];
      if k == i {
        assert sorted[k] == pre[min];
        assert swapped[k] == pre[min];
      } else if k == min {
        assert sorted[k] == pre[i];
        assert swapped[k] == pre[i];
      } else {
        assert sorted[k] == pre[k];
        assert swapped[k] == pre[k];
      }
    }
    assert sorted[..] == swapped;

    ghost var A := pre[..i];
    ghost var B := pre[i + 1..min];
    ghost var C := pre[min + 1..];
    assert pre == A + [pre[i]] + B + [pre[min]] + C;
    assert swapped == A + [pre[min]] + B + [pre[i]] + C;
    calc {
      multiset(swapped);
    ==
      multiset(A + [pre[min]] + B + [pre[i]] + C);
    ==
      multiset(A) + multiset([pre[min]]) + multiset(B) + multiset([pre[i]]) + multiset(C
          ↪ );
    ==
      multiset(A) + multiset([pre[i]]) + multiset(B) + multiset([pre[min]]) + multiset(C
          ↪ );
    ==
      multiset(A + [pre[i]] + B + [pre[min]] + C);
    ==
      multiset(pre);
    }
    assert multiset(sorted[..]) == multiset(pre);
  } else {
    assert sorted[..] == pre;
    assert multiset(sorted[..]) == multiset(pre);
  }

  assert {:split_here} true;
  assert sorted[..] == pre[i := pre[min]][min := pre[i]];
  assert sorted[i] == pre[min];
  assert sorted[min] == pre[i];
```

```
forall k | 0 <= k < n && k != i && k != min
  ensures sorted[k] == pre[k]
{
  assert sorted[..][k] == (pre[i := pre[min]][min := pre[i]])[k];
}
assert multiset(sorted[..]) == multiset(inputs[..]);

forall q | i + 1 <= q < n
  ensures sorted[i] <= sorted[q]
{
  if q == min {
    assert sorted[q] == pre[i];
    assert pre[min] <= pre[i];
  } else {
    assert sorted[q] == pre[q];
    assert pre[min] <= pre[q];
  }
}

forall p, q | 0 <= p < q < i + 1
  ensures sorted[p] <= sorted[q]
{
  if q < i {
    assert sorted[p] == pre[p];
    assert sorted[q] == pre[q];
    assert pre[p] <= pre[q];
  } else {
    assert q == i;
    assert p < i;
    assert sorted[p] == pre[p];
    assert sorted[i] == pre[min];
    assert pre[p] <= pre[min];
  }
}

forall p, q | 0 <= p < i + 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if p < i {
    assert sorted[p] == pre[p];
    if q == min {
      assert sorted[q] == pre[i];
      assert pre[p] <= pre[i];
    } else {
      assert sorted[q] == pre[q];
      assert pre[p] <= pre[q];
    }
  } else {
    assert p == i;
    assert sorted[i] == pre[min];
    if q == min {
```

```
        assert sorted[q] == pre[i];
        assert pre[min] <= pre[i];
      } else {
        assert sorted[q] == pre[q];
        assert pre[min] <= pre[q];
      }
    }
  }

  i := i + 1;
 }

 assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
 assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Total Token Usage

Input tokens: 8605
Output tokens: 21884
Reasoning tokens: 15836
Sum of 'total tokens': 30489

Experiment Timings

Overall Experiment started at 1780748209615, ended at 1780748694375, lasting 484760ms (484.76 seconds)
Iteration #1 started at 1780748209615, ended at 1780748425086, lasting 215471ms (215.47 seconds)
Iteration #2 started at 1780748425086, ended at 1780748511993, lasting 86907ms (86.91 seconds)
Iteration #3 started at 1780748512038, ended at 1780748694374, lasting 182336ms (182.34 seconds)